Machine Learning

Confusion Matrix and Classification Performance Metrics

1. Concept of Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model. It compares the actual (true) class labels with the predicted class labels generated by the model, and shows exactly where the model is getting "confused" between classes.

Why do we need it?

- Accuracy alone can be misleading, especially when the dataset is imbalanced (e.g., 95% of samples belong to one class).
- The confusion matrix breaks down predictions into specific categories (correct and incorrect, for each class), which lets us compute richer metrics like precision, recall, and F1-score.
- It gives a complete picture of what kind of errors the model is making — not just how many.

General structure:

- Rows typically represent the actual class.
- Columns typically represent the predicted class.
- Each cell (i, j) contains the count of instances whose actual class is i and predicted class is j.

2. TP, TN, FP, FN — Definitions with Example

For a binary classification problem, we usually designate one class as Positive (P) and the other as Negative (N).

Term	Full Form	Meaning
TP	True Positive	Model predicted Positive, and the actual class is Positive (correct)
TN	True Negative	Model predicted Negative, and the actual class is Negative (correct)
FP	False Positive	Model predicted Positive, but actual class is Negative (Type I error)
FN	False Negative	Model predicted Negative, but actual class is Positive (Type II error)

Example: Medical Test for a Disease

Suppose a model predicts whether a patient has a disease (Positive = has disease, Negative = no disease).

- TP: Patient actually has the disease, and the model correctly predicts "disease."
- TN: Patient does not have the disease, and the model correctly predicts "no disease."
- FP: Patient does not have the disease, but the model wrongly predicts "disease" (a false alarm).
- FN: Patient actually has the disease, but the model wrongly predicts "no disease" (a dangerous miss).

> ***tip:*** *FN is usually the most dangerous error in medical diagnosis (a sick patient sent home undiagnosed), while FP causes unnecessary worry/tests but is less dangerous. Good moment to discuss precision vs. recall trade-offs.*

3. Confusion Matrix for Binary Classification

For two classes (Positive and Negative), the confusion matrix is a 2×2 table:

	Predicted: Positive	Predicted: Negative
Actual: Positive	TP	FN
Actual: Negative	FP	TN

- Total Positives (P) = TP + FN
- Total Negatives (N) = FP + TN
- Total samples = TP + TN + FP + FN

4. Key Metrics — Definitions and Formulas

a) Accuracy

The proportion of total predictions that were correct.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

b) Error Rate

The proportion of total predictions that were incorrect. It is the complement of accuracy.

$$\text{Error Rate} = (FP + FN) / (TP + TN + FP + FN) = 1 - \text{Accuracy}$$

c) Precision (Positive Predictive Value)

Of all instances predicted as Positive, how many were actually Positive? Measures exactness.

$$\text{Precision} = TP / (TP + FP)$$

d) Recall / Sensitivity (True Positive Rate)

Of all instances that were actually Positive, how many did the model correctly identify? Measures completeness.

$$\text{Recall (Sensitivity)} = TP / (TP + FN)$$

e) Specificity (True Negative Rate)

Of all instances that were actually Negative, how many did the model correctly identify?

$$\text{Specificity} = TN / (TN + FP)$$

f) F1-Score

The harmonic mean of Precision and Recall. Useful when you need a balance between the two, especially with imbalanced classes.

$$\text{F1-Score} = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Quick Reference Table

Metric	Formula	What it Answers
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall, how often is the model correct?
Error Rate	(FP+FN)/(TP+TN+FP+FN)	Overall, how often is the model wrong?
Precision	TP/(TP+FP)	Of predicted positives, how many are correct?
Recall / Sensitivity	TP/(TP+FN)	Of actual positives, how many were caught?
Specificity	TN/(TN+FP)	Of actual negatives, how many were caught?

F1-Score	2·(P·R)/(P+R)	Balance between Precision and Recall

5. Solved Example — Binary Classification

Problem statement:

A spam-detection model is tested on 165 emails. Positive = Spam, Negative = Not Spam. The results are:

- TP = 50 (spam correctly identified as spam)
- TN = 90 (non-spam correctly identified as non-spam)
- FP = 10 (non-spam wrongly marked as spam)
- FN = 15 (spam wrongly marked as non-spam)

Confusion Matrix

	Predicted: Spam	Predicted: Not Spam
Actual: Spam	TP = 50	FN = 15
Actual: Not Spam	FP = 10	TN = 90

Total samples = 50 + 90 + 10 + 15 = 165 ✓

Step-by-step calculation

1. Accuracy

$$\text{Accuracy} = (50 + 90) / 165 = 140/165 = 0.848 = 84.8\%$$

2. Error Rate

$$\text{Error Rate} = (10 + 15) / 165 = 25/165 = 0.152 = 15.2\%$$

3. Precision

$$\text{Precision} = 50 / (50 + 10) = 50/60 = 0.833 = 83.3\%$$

4. Recall (Sensitivity)

$$\text{Recall} = 50 / (50 + 15) = 50/65 = 0.769 = 76.9\%$$

5. Specificity

$$\text{Specificity} = 90 / (90 + 10) = 90/100 = 0.90 = 90\%$$

6. F1-Score

$$F1 = (2 \times 0.833 \times 0.769) / (0.833 + 0.769) = 1.281/1.602 \approx 0.80 = 80\%$$

Interpretation: The model is fairly accurate (84.8%) and quite good at avoiding false alarms on non-spam (specificity 90%), but it misses about 23% of actual spam emails (recall 76.9%), meaning some spam still reaches the inbox.

6. Multi-Class Classification — Definition

Multi-class classification is a classification task where each instance must be assigned to one class out of three or more possible classes (as opposed to binary classification, which has only two classes).

- Each sample belongs to exactly one class among k classes (k > 2).
- Example: Classifying handwritten digits (0–9) → 10 classes
- Example: Classifying an animal image as Cat, Dog, or Rabbit → 3 classes
- Example: Classifying a news article as Sports, Politics, Technology, or Entertainment → 4 classes

> ***Teaching tip:*** *This is different from multi-label classification, where a single instance can belong to more than one class simultaneously (e.g., a movie tagged as both "Comedy" and "Romance").*

7. Multi-Class Confusion Matrix — Understanding with Example

For k classes, the confusion matrix becomes a k × k table, where:

- Rows = actual class
- Columns = predicted class
- Diagonal cells = correctly classified instances for each class
- Off-diagonal cells = misclassifications (which class got confused with which)

Example: Classifying Fruits (Apple, Banana, Orange)

Suppose a model is tested on 90 fruit images (30 of each type). The confusion matrix is:

Actual \ Predicted	Apple	Banana	Orange
Apple	25	3	2
Banana	4	24	2
Orange	1	2	27

Reading the matrix:

- Diagonal values (25, 24, 27): correctly classified instances for Apple, Banana, and Orange respectively.
- Row "Apple": Of 30 actual apples, 25 were correctly predicted as Apple, but 3 were wrongly predicted as Banana and 2 as Orange.
- Column "Apple": Of all images predicted as Apple (25+4+1 = 30), 25 were truly Apple, while 4 were actually Banana and 1 was actually Orange.

Computing per-class metrics (One-vs-Rest approach)

To calculate Precision/Recall for a specific class (e.g., Apple), treat that class as "Positive" and all other classes combined as "Negative":

- TP (Apple) = 25
- FN (Apple) = 3 + 2 = 5 (actual Apple, predicted as something else)
- FP (Apple) = 4 + 1 = 5 (predicted Apple, but actually something else)
- TN (Apple) = 24 + 2 + 2 + 27 = 55 (all remaining correctly-handled non-Apple cases)

```
Precision (Apple) = 25/(25+5) = 0.833      Recall (Apple) = 25/(25+5) = 0.833
```

Overall Accuracy (multi-class)

Accuracy = Sum of diagonal / Total instances = (25+24+27)/90 = 76/90 = 0.844 = 84.4%

> ***Teaching tip:*** *For multi-class problems, per-class Precision/Recall/F1 scores are often combined using macro-averaging (simple average across classes) or weighted-averaging (weighted by class size) to get one overall score for the model.*

Summary Diagram

```
                    PREDICTED
               Positive     Negative
ACTUAL  Positive   TP           FN
        Negative   FP           TN

Accuracy  = (TP+TN) / Total
Precision = TP / (TP+FP)
Recall    = TP / (TP+FN)
F1        = 2PR / (P+R)
```

8. Python Implementation

Python's scikit-learn library makes it easy to compute confusion matrices and all related metrics. Below are two demonstrations that match the worked examples above — one for binary classification (spam example) and one for multi-class classification (fruit example).

a) Binary Classification — Confusion Matrix in Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay,
                             accuracy_score, precision_score,
                             recall_score, f1_score)

# 0 = Not Spam, 1 = Spam
# Built to reproduce TP=50, TN=90, FP=10, FN=15 from Section 5
y_true = [1]*65 + [0]*100
y_pred = [1]*50 + [0]*15 + [1]*10 + [0]*90

# Confusion matrix: rows = actual, columns = predicted
cm = confusion_matrix(y_true, y_pred, labels=[1, 0])
print("Confusion Matrix [Spam, Not Spam]:")
print(cm)

# Extract TP, FN, FP, TN directly from the matrix
TP, FN, FP, TN = cm[0, 0], cm[0, 1], cm[1, 0], cm[1, 1]
print(f"TP={TP}, FN={FN}, FP={FP}, TN={TN}")

# Metrics
accuracy  = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label=1)
recall    = recall_score(y_true, y_pred, pos_label=1)
f1        = f1_score(y_true, y_pred, pos_label=1)
specificity = TN / (TN + FP)

print(f"Accuracy    : {accuracy:.3f}")
print(f"Precision   : {precision:.3f}")
print(f"Recall      : {recall:.3f}")
print(f"Specificity : {specificity:.3f}")
```

```
print(f"F1-Score    : {f1:.3f}")

# Visualize
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=["Spam", "Not Spam"])
disp.plot(cmap="Blues", values_format="d")
plt.title("Binary Confusion Matrix - Spam Detection")
plt.show()
```

Expected output (metrics):

```
TP=50, FN=15, FP=10, TN=90
Accuracy    : 0.848
Precision   : 0.833
Recall      : 0.769
Specificity : 0.900
F1-Score    : 0.800
```

These match the manual calculations from Section 5, confirming the by-hand computation.

b) Multi-Class Classification — Confusion Matrix in Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay,
                             classification_report, accuracy_score)

labels = ["Apple", "Banana", "Orange"]

# Built to reproduce the 3x3 matrix from Section 7
y_true = (["Apple"]*30 + ["Banana"]*30 + ["Orange"]*30)
y_pred = (["Apple"]*25 + ["Banana"]*3 + ["Orange"]*2 +
          ["Apple"]*4  + ["Banana"]*24 + ["Orange"]*2 +
          ["Apple"]*1  + ["Banana"]*2  + ["Orange"]*27)

# Confusion matrix
cm = confusion_matrix(y_true, y_pred, labels=labels)
print("Multi-Class Confusion Matrix:")
print(cm)

# Overall accuracy = sum of diagonal / total
accuracy = accuracy_score(y_true, y_pred)
print(f"Overall Accuracy: {accuracy:.3f}")

# Per-class Precision, Recall, F1 (One-vs-Rest, computed automatically)
print(classification_report(y_true, y_pred, target_names=labels))

# Visualize
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap="Greens", values_format="d")
plt.title("Multi-Class Confusion Matrix - Fruit Classification")
plt.show()
```

Expected output (summary):

```
Overall Accuracy: 0.844

              precision    recall  f1-score   support

       Apple       0.83      0.83      0.83        30
      Banana       0.83      0.80      0.81        30
      Orange       0.87      0.90      0.89        30

    accuracy                           0.84        90
   macro avg       0.84      0.84      0.84        90
weighted avg       0.84      0.84      0.84        90
```

tip: *classification_report() automatically computes macro average (simple mean across classes) and weighted average (weighted by support/class size) — a good moment to explain both averaging strategies to students, and to have them compare the printed values against the manual Apple calculation done by hand in Section 7.*